

12. Machine Learning: Perceptrons and Logistic Regression

Ack: Berkeley AI course

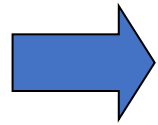
Outline

- Perceptron
- Logistic Regression

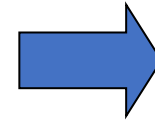
Features

 x $f(x)$ y

```
Hello,  
  
Do you want free print  
cartridges? Why pay more  
when you can get them  
ABSOLUTELY FREE! Just
```



```
# free      : 2  
YOUR_NAME   : 0  
MISSPELLED  : 2  
FROM_FRIEND : 0  
...
```



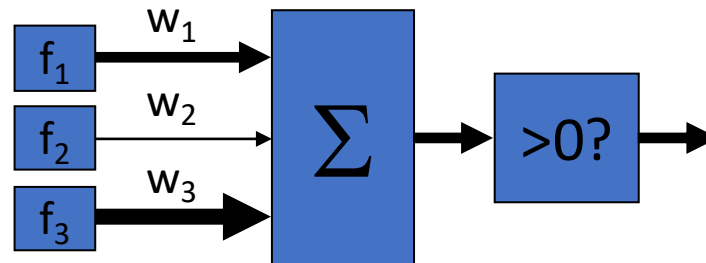
SPAM
or
HAM

Linear Classifiers

- Inputs are **feature values**
- Each feature has a **weight**
- Sum is the **activation**

$$\text{activation}_w(x) = \sum_i w_i \cdot f_i(x) = w \cdot f(x)$$

- If the activation is:
 - Positive, output +1
 - Negative, output -1



Weights

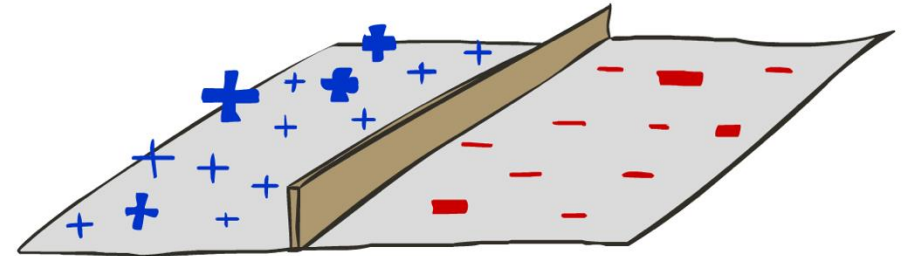
- Binary case: compare features to a weight vector
- Learning: figure out the weight vector from examples

$$\begin{pmatrix} \# \text{ free} & : & 4 \\ \text{YOUR_NAME} & : & -1 \\ \text{MISPELLED} & : & 1 \\ \text{FROM_FRIEND} & : & -3 \\ \dots & & \end{pmatrix}$$
 w $f(x_1)$
$$\begin{pmatrix} \# \text{ free} & : & 2 \\ \text{YOUR_NAME} & : & 0 \\ \text{MISPELLED} & : & 2 \\ \text{FROM_FRIEND} & : & 0 \\ \dots & & \end{pmatrix}$$
 $f(x_2)$
$$\begin{pmatrix} \# \text{ free} & : & 0 \\ \text{YOUR_NAME} & : & 1 \\ \text{MISPELLED} & : & 1 \\ \text{FROM_FRIEND} & : & 1 \\ \dots & & \end{pmatrix}$$

*Dot product $w \cdot f$ positive
means the positive class*

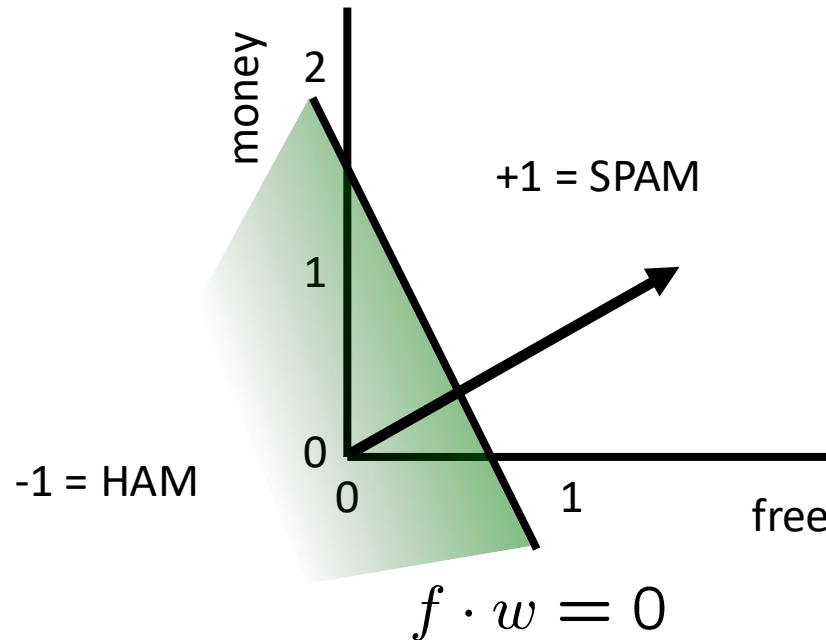
Binary Decision Rule

- In the space of feature vectors
 - Examples are points
 - A weight vector corresponds to a hyperplane
 - One side corresponds to $Y=+1$
 - Other corresponds to $Y=-1$



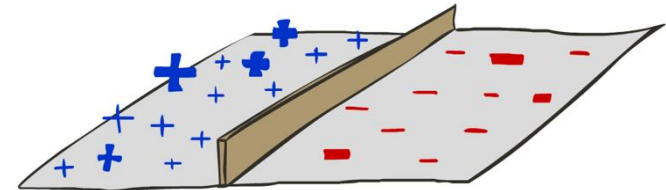
w

BIAS	:	-3
free	:	4
money	:	2



Learning: Binary Perceptron

- Figure out the weight vector from training data
 - Start with weights = 0
 - For each training instance:
 - Classify with current weights
 - If correct (i.e., $y=y^*$), no change!
 - If wrong: adjust the weight vector



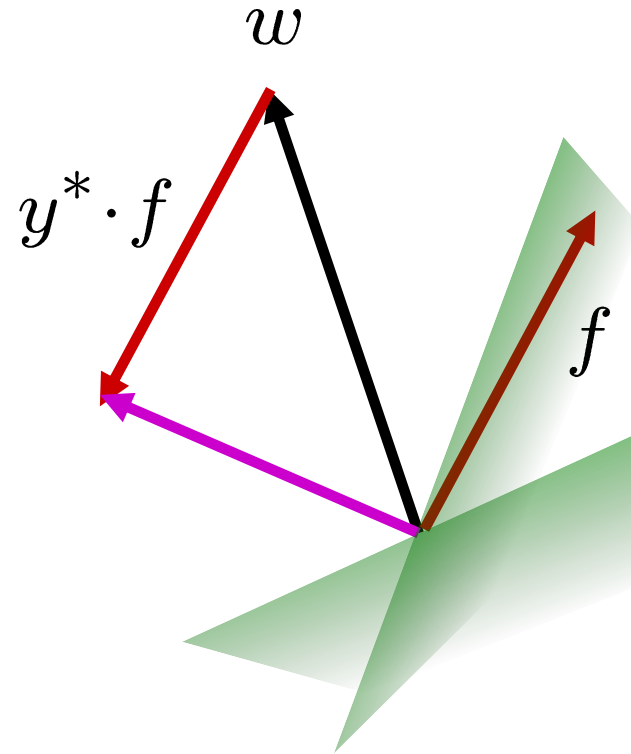
Learning: Binary Perceptron

- Start with weights = 0
- For each training instance:
 - Classify with current weights

$$y = \begin{cases} +1 & \text{if } w \cdot f(x) \geq 0 \\ -1 & \text{if } w \cdot f(x) < 0 \end{cases}$$

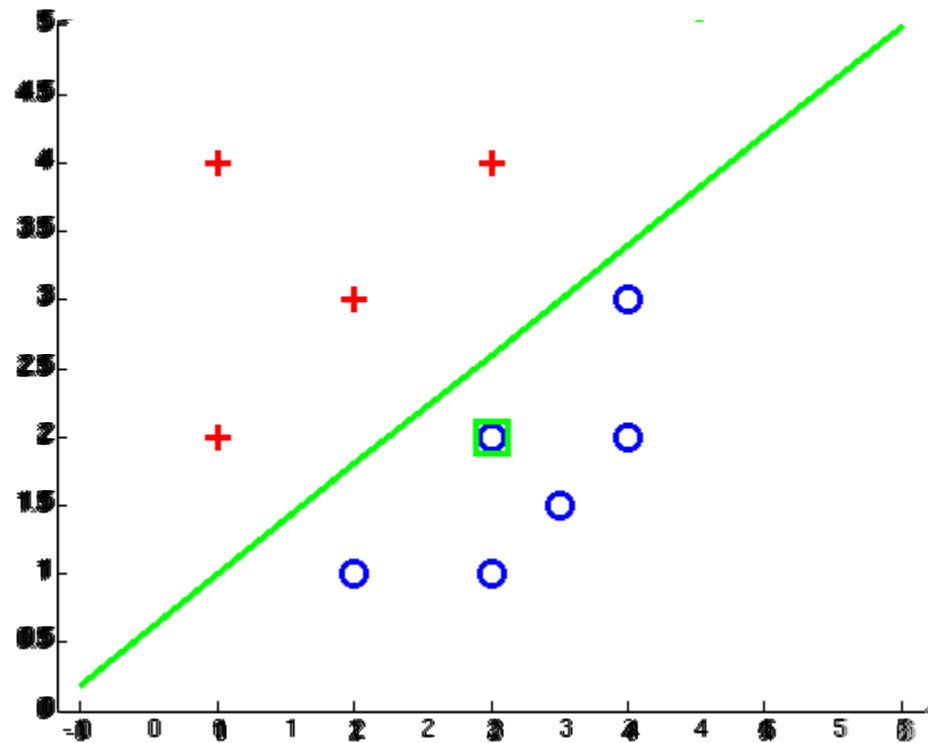
- If correct (i.e., $y=y^*$), no change!
- If wrong: adjust the weight vector by adding or subtracting the feature vector. Subtract if y^* is -1.

$$w = w + y^* \cdot f$$



Examples: Perceptron

- Separable Case



Multiclass Decision Rule

- If we have multiple classes:
 - A weight vector for each class:

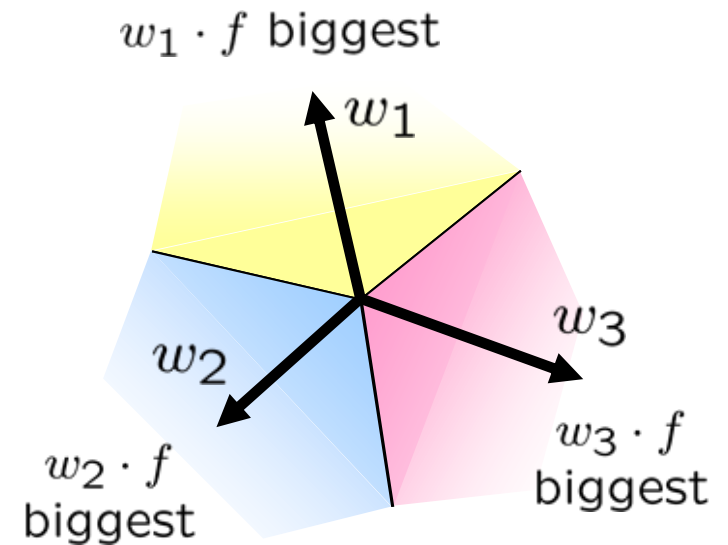
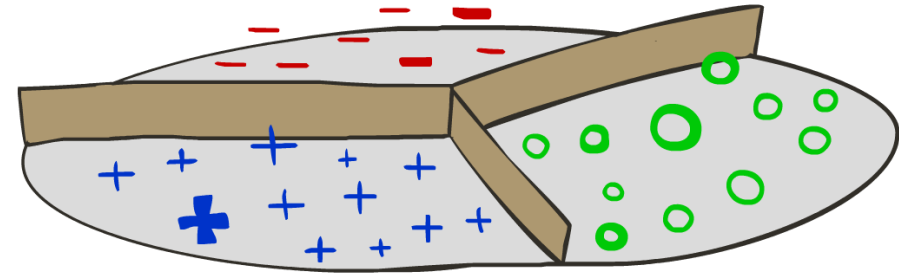
$$w_y$$

- Score (activation) of a class y :

$$w_y \cdot f(x)$$

- Prediction highest score wins

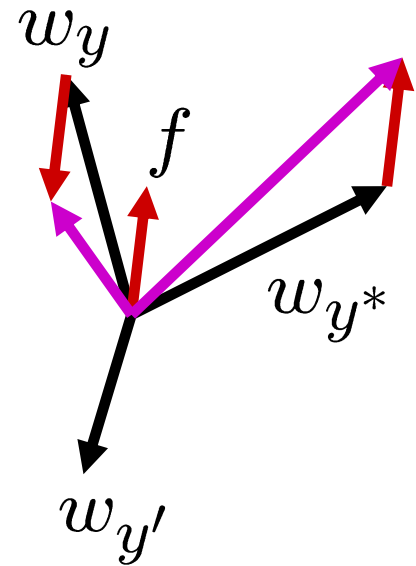
$$y = \arg \max_y w_y \cdot f(x)$$



Learning: Multiclass Perceptron

- Start with all weights = 0
- For each training instance:
 - Predict with current weights
$$y = \arg \max_y w_y \cdot f(x)$$
 - If correct, no change!
 - If wrong: lower score of wrong answer, raise score of right answer

$$w_y = w_y - f(x)$$
$$w_{y^*} = w_{y^*} + f(x)$$



Example: Multiclass Perceptron

Training instance	$f(x)$	w_{SPORTS}	$w_{POLITICS}$	w_{TECH}
“win the vote”	[1 1 0 1 1]	BIAS : 1 win : 0 game : 0 vote : 0 the : 0	BIAS : 0 win : 0 game : 0 vote : 0 the : 0	BIAS : 0 win : 0 game : 0 vote : 0 the : 0
“win the election”	[1 1 0 0 1]			
“win the game”	[1 1 1 0 1]			

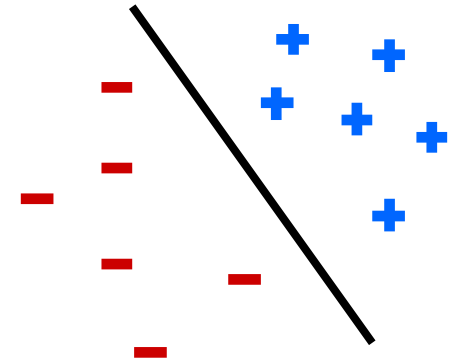
Training	$f(x)$ {BIAS, “win”, “game”, “vote”, “the”}	Prediction Before Update	SPORTS	POLITICS	Tech
Initial W			[1, 0, 0, 0, 0]	[0, 0, 0, 0, 0]	[0, 0, 0, 0, 0]
“win the vote”	[1, 1, 0, 1, 1]	SPORTS	[0, -1, 0, -1, -1]	[1, 1, 0, 1, 1]	[0, 0, 0, 0, 0]
“win the election”	[1, 1, 0, 0, 1]	POLITICS	[0, -1, 0, -1, -1]	[1, 1, 0, 1, 1]	[0, 0, 0, 0, 0]
“win the game”	[1, 1, 1, 0, 1]	POLITICS	[1, 0, 1, -1, 0]	[0, 0, -1, 1, 0]	[0, 0, 0, 0, 0]

Properties of Perceptrons

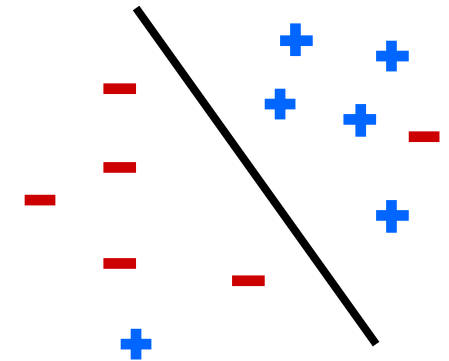
- Separability: data is separable if some parameters get the training set perfectly correct
- Convergence: if the training is separable, binary perceptron will eventually converge
- Mistake Bound: the maximum number of mistakes (binary case) related to the *margin* or degree of separability

$$\text{mistakes} < \frac{k}{\delta^2}$$

Separable

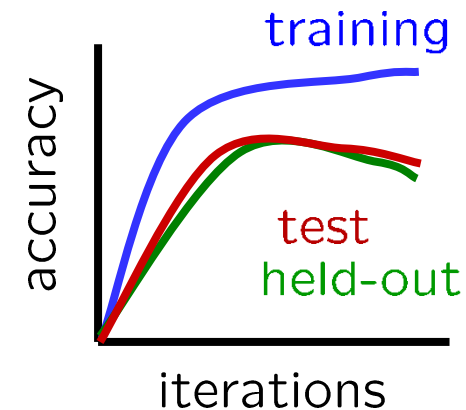
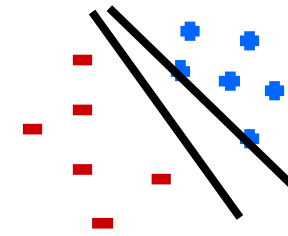
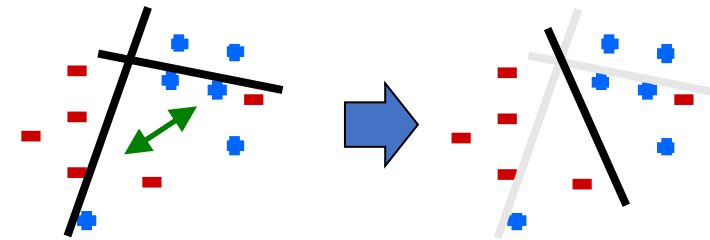


Non-Separable



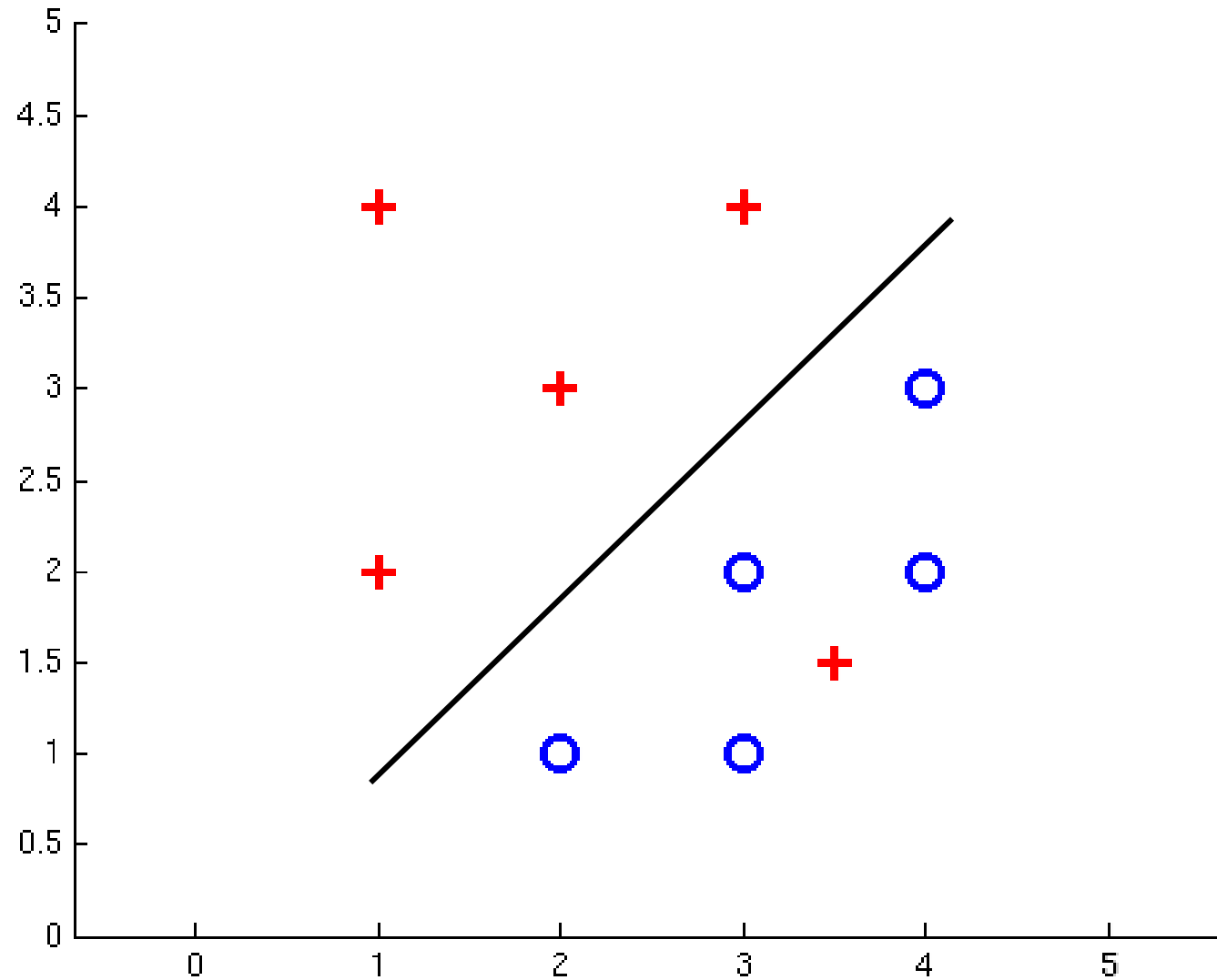
Problems with the Perceptron

- Noise: if the data isn't separable, weights might thrash
 - Averaging weight vectors over time can help (averaged perceptron)
- Mediocre generalization: finds a “barely” separating solution
- Overtraining: test / held-out accuracy usually rises, then falls
 - Overtraining is a kind of overfitting

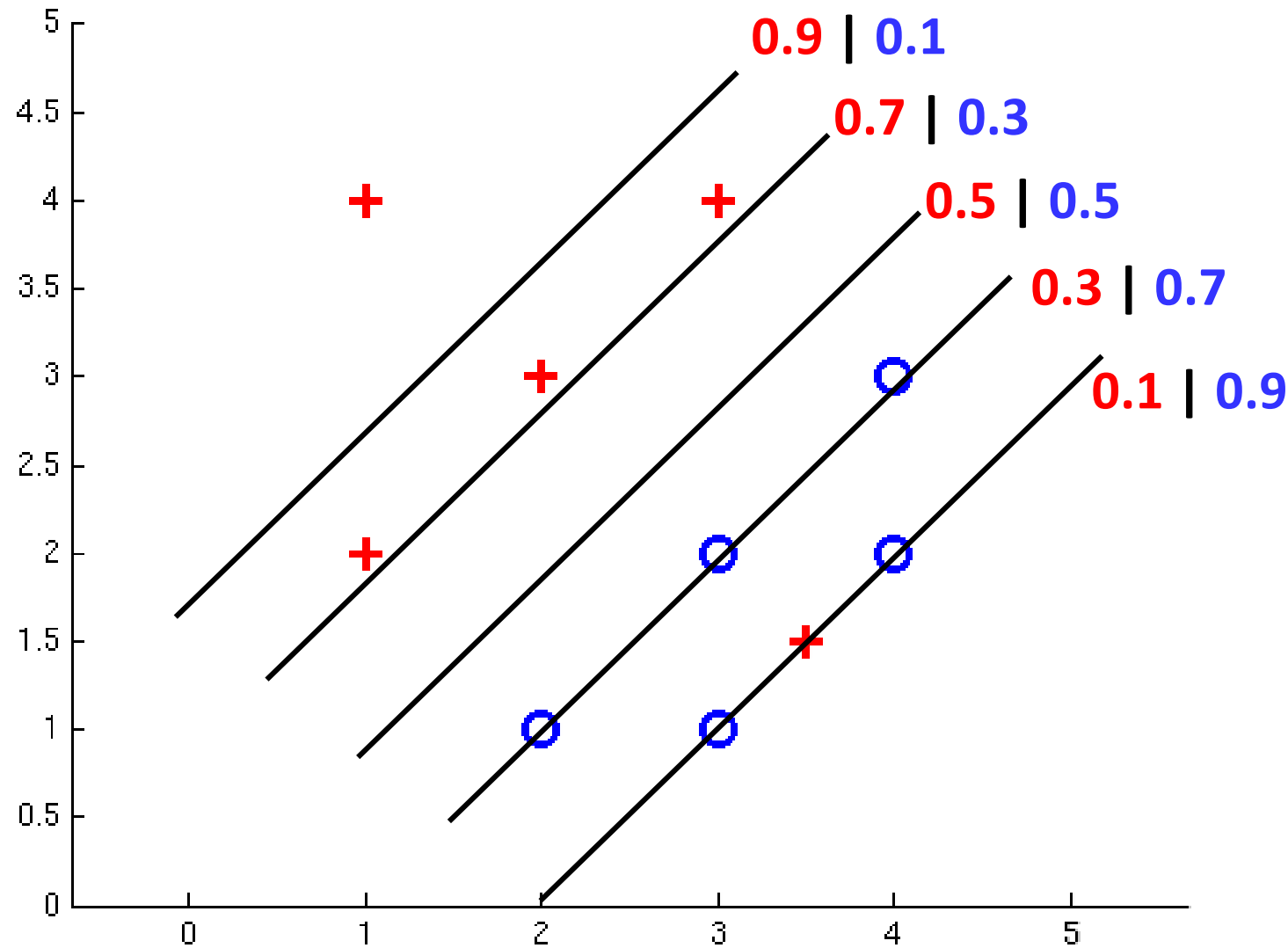


Non-Separable Case: Deterministic Decision

Even the best linear boundary makes at least one mistake



Non-Separable Case: Probabilistic Decision



Outline

- Perceptron
- **Logistic Regression**

How to get probabilistic decisions?

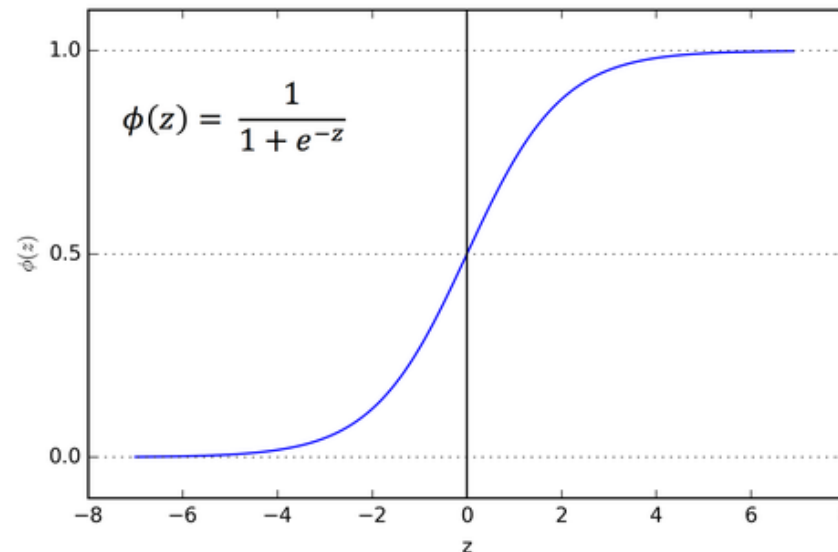
$$z = w \cdot f(x)$$

- Perceptron scoring:

- If $z = w \cdot f(x)$ very positive $\rightarrow$ want probability going to 1
- If $z = w \cdot f(x)$ very negative $\rightarrow$ want probability going to 0

- Sigmoid function

$$\phi(z) = \frac{1}{1 + e^{-z}}$$



Best w ?

- Maximum likelihood estimation:

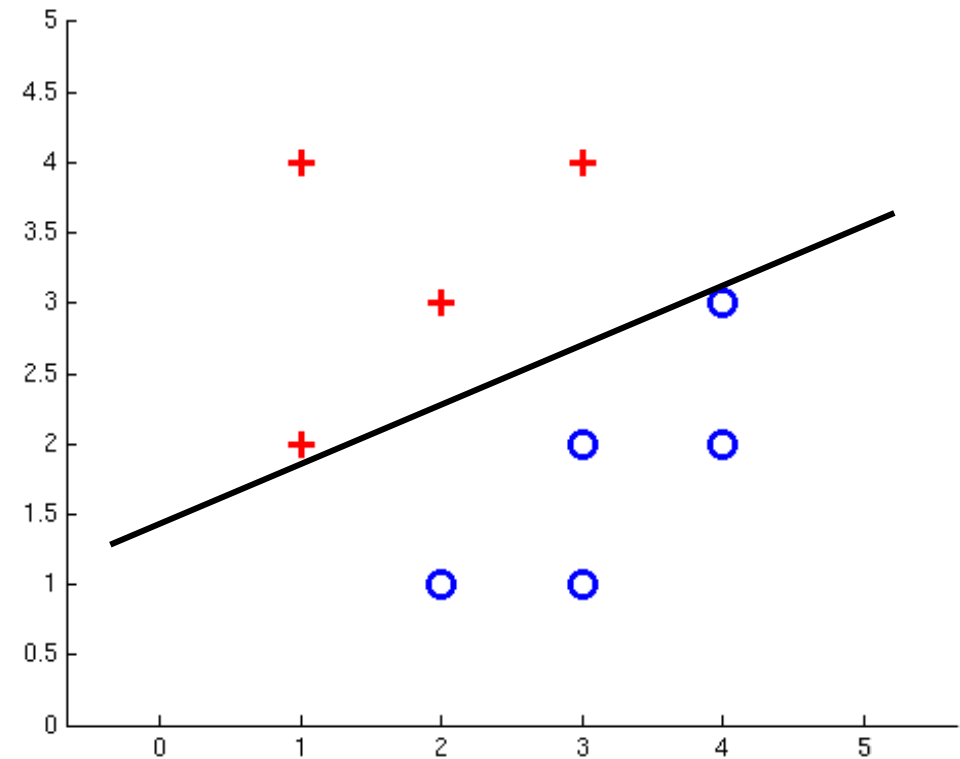
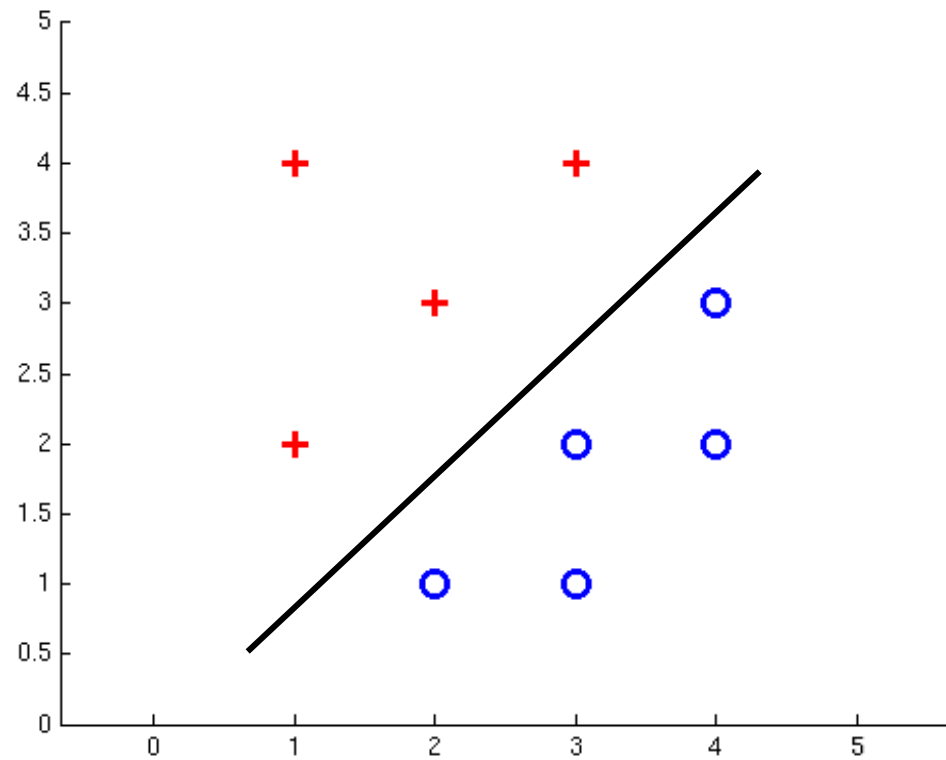
with:
$$\max_w ll(w) = \max_w \sum_i \log P(y^{(i)} | x^{(i)}; w)$$

$$P(y^{(i)} = +1 | x^{(i)}; w) = \frac{1}{1 + e^{-w \cdot f(x^{(i)})}}$$

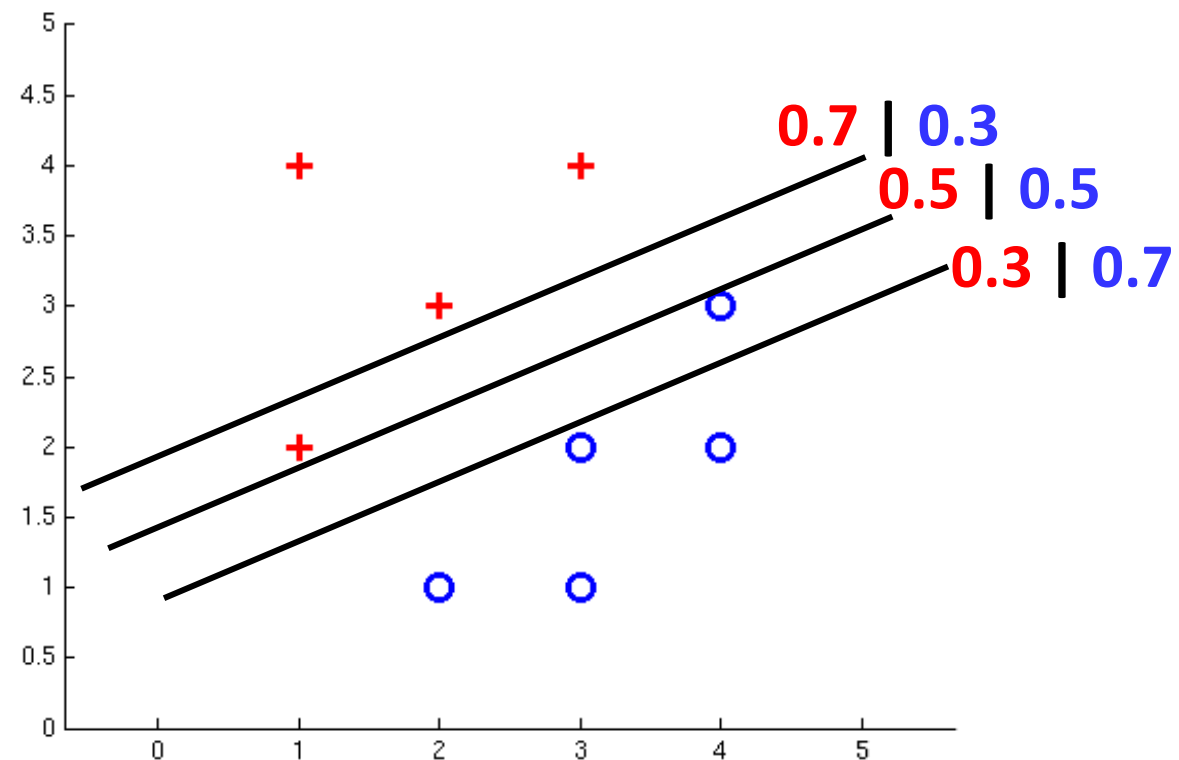
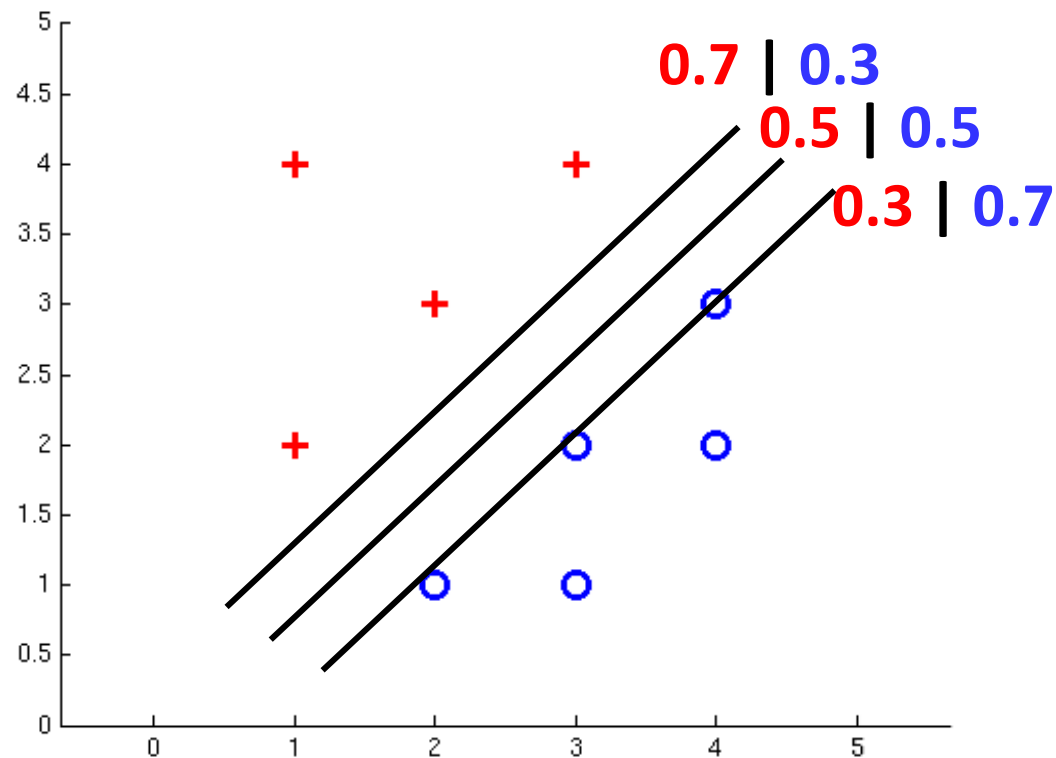
$$P(y^{(i)} = -1 | x^{(i)}; w) = 1 - \frac{1}{1 + e^{-w \cdot f(x^{(i)})}}$$

= Logistic Regression

Separable Case: Deterministic Decision – Many Options



Separable Case: Probabilistic Decision – Clear Preference



$$\max_w ll(w) = \max_w \sum_i \log P(y^{(i)} | x^{(i)}; w)$$

Multiclass Logistic Regression

- Recall Perceptron:

- A weight vector for each class:

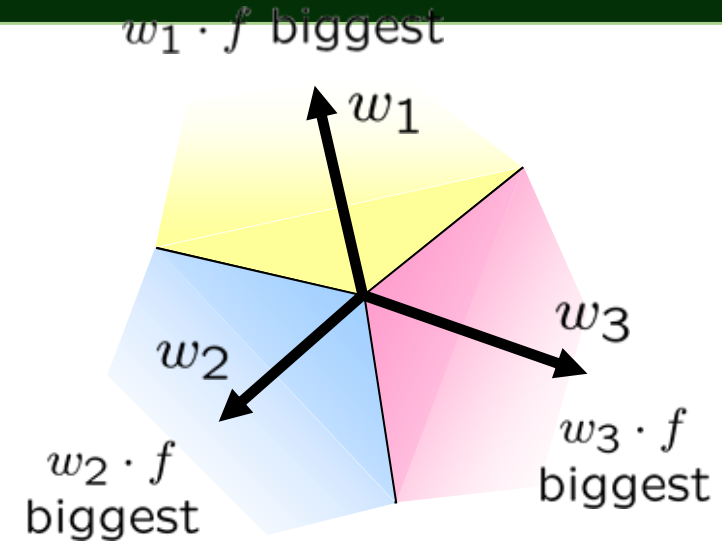
$$w_y$$

- Score (activation) of a class y :

$$z = w \cdot f(x)$$

- Prediction highest score wins

$$y = \arg \max_y w_y \cdot f(x)$$



- How to make the scores into probabilities?

$$\underbrace{z_1, z_2, z_3}_{\text{original activations}} \rightarrow \underbrace{\frac{e^{z_1}}{e^{z_1} + e^{z_2} + e^{z_3}}, \frac{e^{z_2}}{e^{z_1} + e^{z_2} + e^{z_3}}, \frac{e^{z_3}}{e^{z_1} + e^{z_2} + e^{z_3}}}_{\text{softmax activations}}$$

Best w ?

- Maximum likelihood estimation:

$$\max_w ll(w) = \max_w \sum_i \log P(y^{(i)} | x^{(i)}; w)$$

with:

$$P(y^{(i)} | x^{(i)}; w) = \frac{e^{w_{y^{(i)}} \cdot f(x^{(i)})}}{\sum_y e^{w_y \cdot f(x^{(i)})}}$$

= Multi-Class Logistic Regression

Cross-entropy loss

Cross-entropy loss function (binary classification)

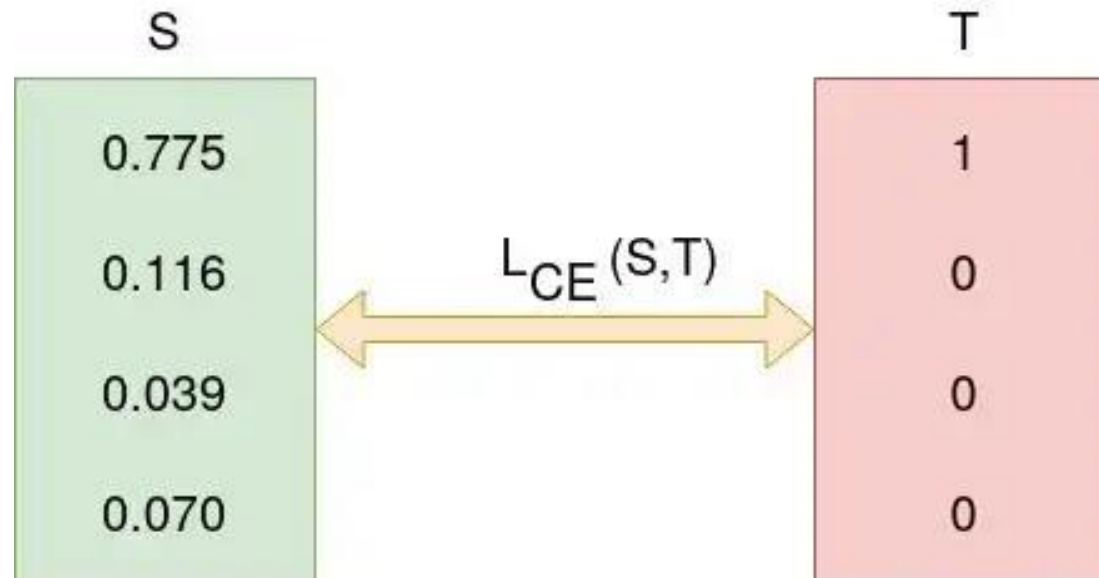
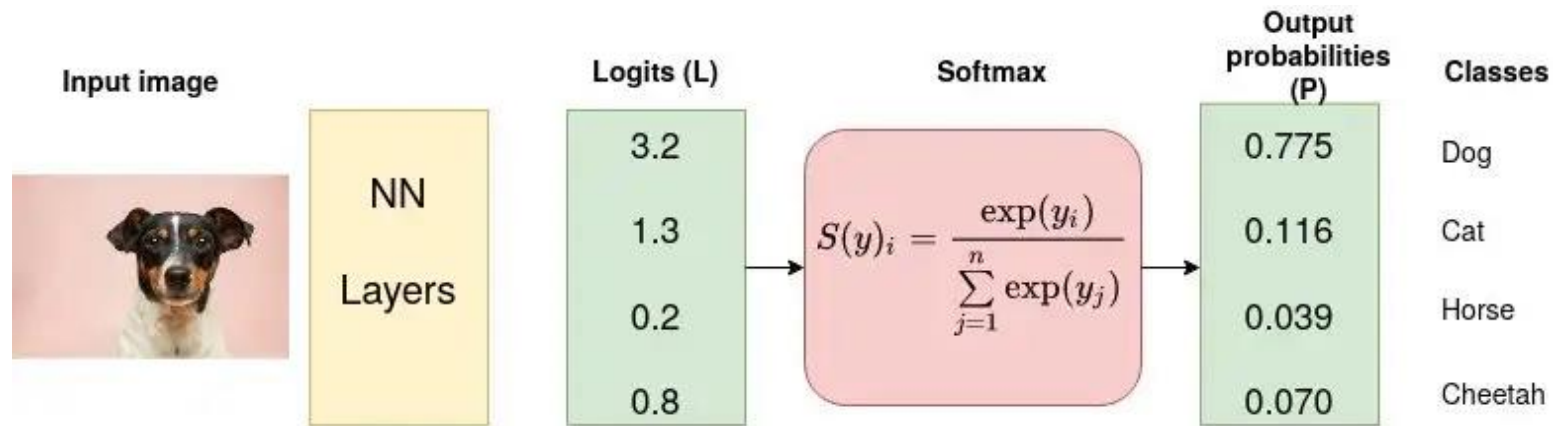
- $J = - \{t \log(y) + (1-t) \log(1-y)\}$, where
t = target label
y = predicted label (from the logistic regression)
- For all the training examples,

$$J = - \sum_{n=1}^N t_n \log(y_n) + (1 - t_n) \log(1 - y_n)$$

- Compare with mean squared loss:

$$J = \sum_{n=1}^N (t_n - y_n)^2$$

Cross-entropy loss



$$\begin{aligned}L_{CE} &= - \sum_{i=1} T_i \log(S_i) \\&= - [1 \log_2(0.775) + 0 \log_2(0.126) + 0 \log_2(0.039) + 0 \log_2(0.070)] \\&= - \log_2(0.775) \\&= 0.3677\end{aligned}$$